

2주차 예습과제-딥러닝 파이토치 교과서 4장

4장-딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

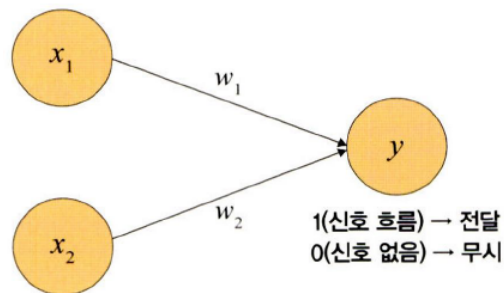
📌 퍼셉트론(Perceptron)

- **정의:** 오늘날 인공 신경망에서 이용하는 구조(입력층, 출력층, 가중치로 구성된 구조)
- **고안자:** 프랭크 로젠블라트(Frank Rosenblatt), 1957년
- **분류:** 선형 분류기
- **의의:** 오늘날 신경망(딥러닝)의 기원이 되는 알고리즘

동작 원리

- 다수의 신호(흐름이 있는)를 입력으로 받아 **하나의 신호를 출력**
- 이 신호를 입력으로 받아 '**흐른다 / 안 흐른다**' (1 또는 0) 정보를 앞으로 전달

▼ 그림 4-1 퍼셉트론 원리



🔍 퍼셉트론 원리

- 입력: x_1, x_2
- 가중치: w_1, w_2
- 출력: y
- 1 (신호 흐름) → 전달
- 0 (신호 없음) → 무시

→ 입력이 두 개(x_1, x_2) 있다고 할 때, 컴퓨터가 논리적으로 인식하는 방식을 알아보기 위해 **논리 게이트**로 확인

◆ AND 게이트

- 모든 입력이 '1'일 때 작동
- 입력 중 어떤 하나라도 '0'을 갖는다면 작동을 멈춤

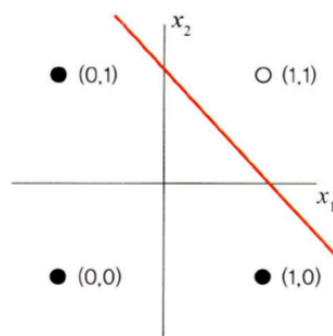
표 4-1 AND 게이트 진리표

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

🔍 그림 4-2 AND 게이트

- (0,0), (1,0), (0,1) → 검은색 점 ($y = 0$)
- (1,1) → 흰색 점 ($y = 1$)
- 하나의 직선으로 검은색 점과 흰색 점을 분류할 수 있음 → **선형 분리 가능**

▼ 그림 4-2 AND 게이트



◆ OR 게이트

- 입력에서 둘 중 하나만 '1'이거나 둘 다 '1'일 때 작동
- 입력 모두가 '0'을 갖는 경우를 제외한 나머지가 모두 '1' 값을 가짐

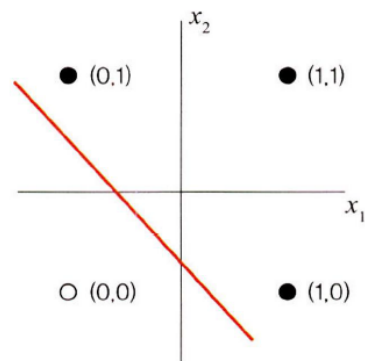
표 4-2 OR 게이트 진리표

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

🔍 그림 4-3 OR 게이트

- $(0,0) \rightarrow$ 흰색 점 ($y = 0$)
- $(1,0), (0,1), (1,1) \rightarrow$ 검은색 점 ($y = 1$)
- AND 게이트와 마찬가지로 **하나의 직선**으로 분류 가능 $\rightarrow$ 선형 분리 가능

▼ 그림 4-3 OR 게이트



◆ XOR 게이트

- 배타적 논리합
- 입력 두 개 중 한 개만 '1'일 때 작동하는 논리 연산

표 4-3 XOR 게이트 진리표

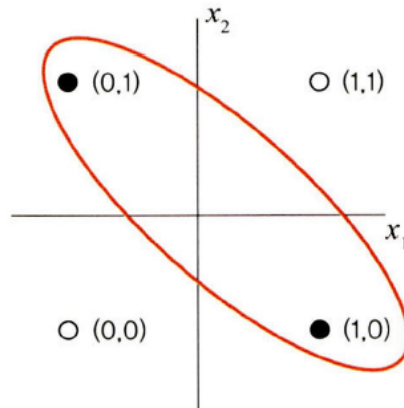
x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

🔍 그림 4-4 XOR 게이트

- $(0,0), (1,1) \rightarrow$ 흰색 점 ($y = 0$)

- $(0,1), (1,0) \rightarrow$ 검은색 점 ($y = 1$)
- 검은색 점과 흰색 점을 하나의 직선으로 나눌 수 없음 $\rightarrow$ 비선형 분리

♥ 그림 4-4 XOR 게이트



! XOR 게이트의 문제

- 데이터가 **비선형적으로** 분리되기 때문에 제대로 된 분류가 어려움
- 단층 퍼셉트론은 AND, OR 연산에 대해서는 학습 가능
- 하지만 **XOR에 대해서는 학습 불가능**

📌 다층 퍼셉트론(Multi-Layer Perceptron, MLP)

- XOR 문제를 극복하기 위해 고안됨
- *입력층과 출력층 사이에 하나 이상의 중간층(은닉층)**을 두어 **비선형적으로** 분리되는 데이터에 대해서도 학습 가능하도록 함

📌 심층 신경망(Deep Neural Network, DNN)

- 입력층과 출력층 사이에 **은닉층이 여러 개** 있는 신경망
- 심층 신경망을 다른 이름으로 **딥러닝**이라고 함

✅ 한눈에 정리

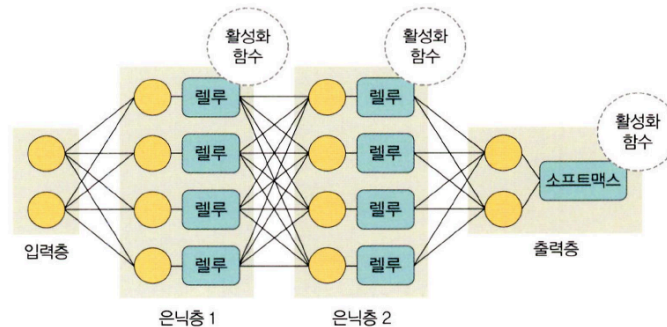
구분	작동 조건	선형 분리	단층 퍼셉트론 학습
AND	모든 입력이 1	가능	✅ 가능
OR	하나라도 1	가능	✅ 가능
XOR	둘 중 하나만 1	불가능	❌ 불가능

4.2 딥러닝 구조

4.2.1 딥러닝 용어

- 딥러닝은 **입력층, 출력층, 두 개 이상의 은닉층**으로 구성
- 입력 신호를 전달하기 위해 **다양한 함수**도 사용

▼ 그림 4-5 딥러닝 구조



- 입력층 → 은닉층 1 (렐루) → 은닉층 2 (렐루) → 출력층 (소프트맥스)
- 각 은닉층·출력층마다 **활성화 함수**가 붙음
- 은닉층: **렐루(ReLU)** / 출력층: **소프트맥스(Softmax)**

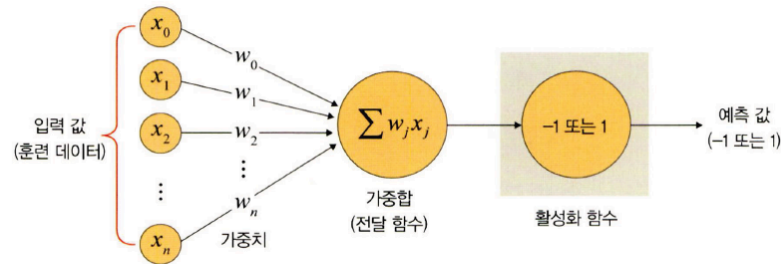
딥러닝 구성 요소

구분	구성 요소	설명
층	입력층 (input layer)	데이터를 받아들이는 층
층	은닉층 (hidden layer)	모든 입력 노드부터 입력 값을 받아 가중합을 계산하고, 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
층	출력층 (output layer)	신경망의 최종 결괏값이 포함된 층
-	가중치 (weight)	노드와 노드 간 연결 강도
-	바이어스 (bias)	가중합에 더해 주는 상수로, 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
-	가중합 (weighted sum), 전달 함수	가중치와 신호의 곱을 합한 것
함수	활성화 함수 (activation function)	신호를 입력받아 이를 적절히 처리하여 출력해 주는 함수
함수	손실 함수 (loss function)	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수

📌 가중치 (Weight)

- 입력 값이 연산 결과에 미치는 영향력을 조절하는 요소
- 예: w_1 값이 0 혹은 0과 가까운 0.001이라면, x_1 이 아무리 큰 값이라도 $x_1 \times w_1$ 값은 0이거나 0에 가까운 값이 됨
- → 입력 값의 연산 결과를 조정하는 역할

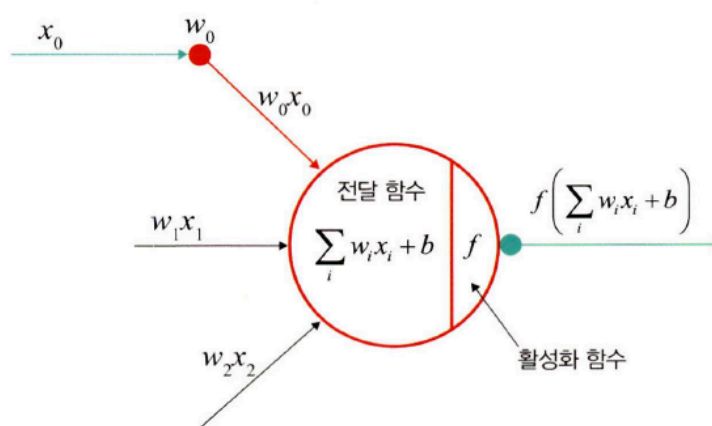
▼ 그림 4-6 가중치



📌 가중합 (Weighted Sum) 또는 전달 함수 (Transfer Function)

- 각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달
- 이 값들을 모두 더한 합계 = 가중합
- 노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 → **전달 함수 (transfer function)**라고도 함

▼ 그림 4-7 전달 함수



가중합 공식

$\sum w_i x_i + b$ (w: 가중치, b: 바이어스)

📌 활성화 함수 (Activation Function)

- 전달 함수에서 전달받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

- 비선형 함수: 직선으로 표현할 수 없는 데이터 사이의 관계를 표현하는 함수
- 종류: 시그모이드(sigmoid), 하이퍼볼릭 탄젠트(hyperbolic tangent), 렐루(ReLU) 등

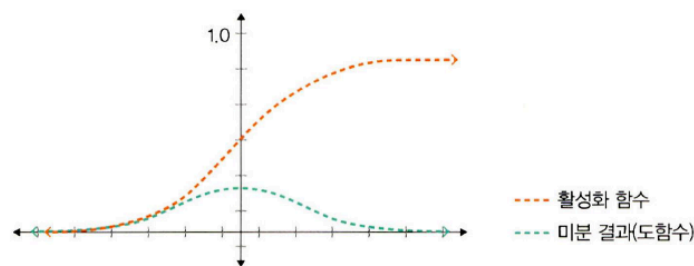
◆ 시그모이드 함수 (Sigmoid)

- 선형 함수의 결과를 0 ~ 1 사이에서 비선형 형태로 변형
- 주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는 데 사용
- 과거에는 인기가 많았으나, 딥러닝 모델의 깊이가 깊어지면 기울기가 사라지는 **기울기 소멸 문제(vanishing gradient problem)**가 발생 → 딥러닝 모델에서는 잘 사용하지 않음
 - 기울기 소멸 문제: 활성화 함수 계산 과정에서 도함수 값이 계속 곱해지면서 가중치 곱댓값이 0이 되어 더 이상 계산이 불가능한 상태

수식

$$f(x) = 1 / (1 + e^{-x})$$

▼ 그림 4-8 시그모이드 활성화 함수와 미분 결과

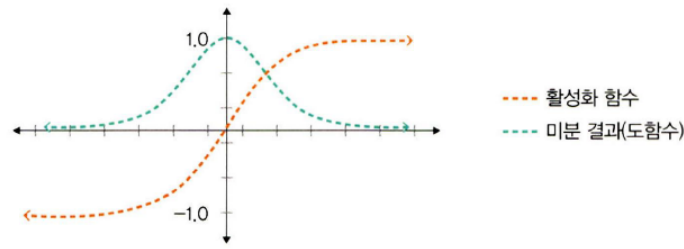


- 활성화 함수: 0 → 1.0으로 완만하게 올라가는 S자 곡선
- 도함수: 0 근처에서 최대, 양쪽으로 갈수록 0에 가까워짐 → 기울기 소멸의 원인

◆ 하이퍼볼릭 탄젠트 함수 (tanh)

- 선형 함수의 결과를 -1 ~ 1 사이에서 비선형 형태로 변형
- 시그모이드에서 곱댓값의 평균이 0이 아닌 양수로 편향된 문제를 해결하는 데 사용
- 하지만 기울기 소멸 문제는 여전히 발생

▼ 그림 4-9 하이퍼볼릭 탄젠트 활성화 함수와 미분 결과

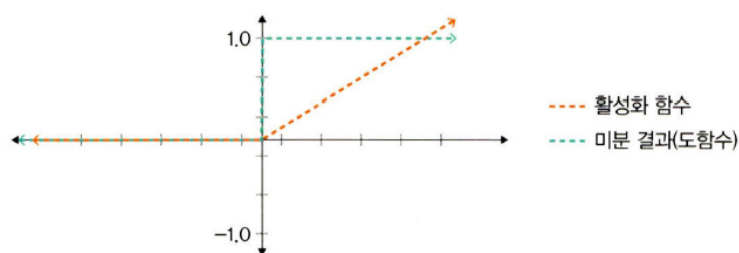


- 활성화 함수: $-1.0 \rightarrow 1.0$ 으로 올라가는 S자 곡선 (원점 기준 대칭)
- 도함수: 0에서 최대 1.0, 양쪽으로 갈수록 0에 가까워짐

◆ 렐루 함수 (ReLU)

- 최근 활발히 사용되는 함수
- 입력(x)이 음수일 때는 0을 출력, 양수일 때는 x 를 출력
- 장점
 - 경사 하강법(gradient descent)에 영향을 주지 않아 학습 속도가 빠름
 - 기울기 소멸 문제가 발생하지 않음
 - 하이퍼볼릭 탄젠트 함수 대비 학습 속도가 **6배** 빠름
- 일반적으로 은닉층에서 사용
- 문제점: 음수 값을 입력받으면 항상 0을 출력하기 때문에 학습 능력이 감소 → 이를 해결하려고 리키 렐루(Leaky ReLU) 함수 등을 사용

▼ 그림 4-10 렐루 활성화 함수와 미분 결과



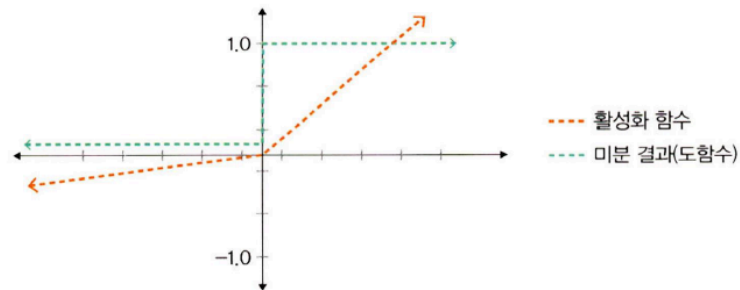
- 활성화 함수: $x < 0$ 에서 0, $x > 0$ 에서 기울기 1인 직선
- 도함수: $x < 0$ 에서 0, $x > 0$ 에서 1.0

◆ 리키 렐루 함수 (Leaky ReLU)

- 입력 값이 음수이면 0이 아닌 0.001처럼 매우 작은 수를 반환

- 입력 값이 수렴하는 구간이 제거되어 → **렐루 함수를 사용할 때 생기는 문제를 해결**

▼ 그림 4-11 리키 렐루 활성화 함수와 미분 결과



- 활성화 함수: $x < 0$ 에서 아주 완만한 기울기의 직선, $x > 0$ 에서 기울기 1인 직선
- 도함수: $x < 0$ 에서 0에 가까운 작은 양수, $x > 0$ 에서 1.0

◆ 소프트맥스 함수 (Softmax)

- 입력 값을 **0 ~ 1** 사이에 출력되도록 정규화하여 **출력 값들의 총합이 항상 1**이 되도록 함
- 보통 딥러닝에서 **출력 노드의 활성화 함수**로 많이 사용

수식

$$y_k = \exp(a_k) / \sum_{i=1}^n \exp(a_i)$$

- $\exp(x)$: 지수 함수(exponential function) — 앞의 식에서는 $\exp(a_k)$ 와 $\exp(a_i)$ 를 의미
- n : 출력층의 뉴런 개수
- y_k : 그중 k 번째 출력
- → **분자**: 입력 신호 a_k 의 지수 함수 / **분모**: 모든 입력 신호의 지수 함수 합

함수	출력 범위	주 용도	문제점
Sigmoid	0 ~ 1	로지스틱 회귀 등 분류(확률 표현)	기울기 소멸, 평균이 0이 아님
tanh	-1 ~ 1	시그모이드의 평균 편향 문제 해결	기울기 소멸 여전히 발생
ReLU	0 ~ x	은닉층 (학습 빠름, 기울기 소멸 X)	음수 입력 시 항상 0 → 학습 능력 감소
Leaky ReLU	아주 작은 음수 ~ x	렐루의 문제 보완	—
Softmax	0 ~ 1 (합 = 1)	출력층 (다중 분류)	—

손실 함수 (Loss Function)

경사 하강법 (Gradient Descent)

- *학습률(η , learning rate)**과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트하는 방법
 - 학습률: 한 번 학습할 때 얼마큼 변화를 주는지에 대한 상수
- 즉, 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법
- 이때 오차를 구하는 방법이 손실 함수

손실 함수

- 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표
- 값이 클수록 많이 틀렸다는 의미 / 값이 '0'에 가까우면 완벽하게 추정할 수 있다는 의미
- 대표적인 손실 함수: 평균 제곱 오차(Mean Squared Error, MSE), 크로스 엔트로피 오차(Cross Entropy Error, CEE)

◆ 평균 제곱 오차 (MSE)

- 실제 값과 예측 값의 차이(error)를 제공하여 평균을 낸 것
- 실제 값과 예측 값의 차이가 클수록 MSE 값도 커짐 → 반대로 이 값이 작을수록 예측력이 좋다는 의미
- 주로 회귀에서 손실 함수로 사용

수식

$$MSE = (1/n) \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- $\hat{y}_i$: 신경망의 출력 (신경망이 추정한 값)
- y_i : 정답 레이블
- i : 데이터의 차원 개수

◆ 크로스 엔트로피 오차 (CEE)

- 분류(classification) 문제에서 **원-핫 인코딩(one-hot encoding)**했을 때만 사용할 수 있는 오차 계산법
- 왜 MSE 대신 사용하는가?
 - 일반적으로 분류 문제에서는 데이터의 출력을 0과 1로 구분하기 위해 시그모이드 함수를 사용

- 시그모이드 함수에 포함된 **자연 상수 e** 때문에 MSE를 적용하면 ****매끄럽지 못한 그래프(울퉁불퉁한 그래프)****가 출력됨
- 이 손실 함수를 적용할 경우 경사 하강법 과정에서 학습이 **지역 최소점에서 멈출 수 있음**
- → 이것을 방지하고자 자연 상수 e에 반대되는 **자연 로그를 모델의 출력 값에 취함**

수식

$$\text{CrossEntropy} = -\sum_{i=1}^n y_i \log \hat{y}_i$$

- $\hat{y}_i$: 신경망의 출력 (신경망이 추정한 값)
- y_i : 정답 레이블
- i : 데이터의 차원 개수

손실 함수	계산 방식	주 용도	파이토치
MSE	오차의 제곱 평균	회귀	<code>torch.nn.MSELoss()</code>
CEE	$-\sum \text{정답} \times \log(\text{예측})$	분류 (원-핫 인코딩)	<code>nn.CrossEntropyLoss()</code>

4.2.2 딥러닝 학습

참고: 원-핫 인코딩 (One-Hot Encoding)

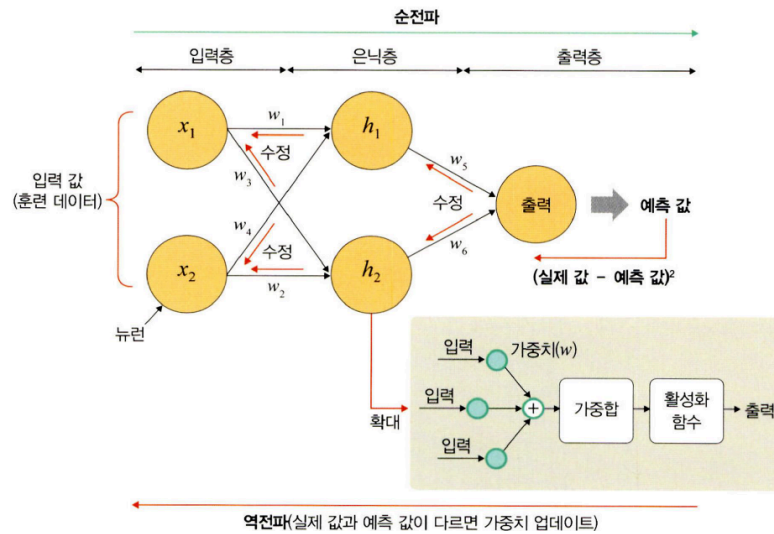
- 단어 집합의 크기를 **벡터 차원**으로 함
- 표현하고 싶은 단어의 인덱스에 **1** 값을 부여한 후, 다른 인덱스에는 **0**을 부여하는 단어의 벡터 표현 방식

예시: (나는, 딥러닝을, 학습하고, 있다)

단어	나는	딥러닝을	학습하고	있다
나는	1	0	0	0
딥러닝을	0	1	0	0
학습하고	0	0	1	0
있다	0	0	0	1

- 딥러닝 학습은 크게 **순전파**와 **역전파**라는 두 단계로 진행

▼ 그림 4-12 순전파와 역전파



- 순전파 방향: 입력층 → 은닉층 → 출력층 (초록 화살표, 왼쪽 → 오른쪽)
- 입력 값(훈련 데이터) x_1, x_2 → 가중치 $w_1 \sim w_4$ → 은닉층 h_1, h_2 → 가중치 w_5, w_6 → 출력 → 예측 값
- 출력에서 **(실제 값 - 예측 값)²** 계산
- 역전파 방향: 출력층 → 은닉층 → 입력층 (빨간 화살표, 오른쪽 → 왼쪽), 각 가중치 옆에 "수정" 표시
- 뉴런 확대: 입력 $\times$ 가중치(w) $\rightarrow \oplus \rightarrow$ 가중합 $\rightarrow$ 활성화 함수 $\rightarrow$ 출력
- 역전파 = 실제 값과 예측 값이 다르면 가중치 업데이트

◆ 1단계: 순전파 (Feedforward)

- 네트워크에 **훈련 데이터가 들어올 때** 발생
- 데이터를 기반으로 **예측 값을 계산**하기 위해 전체 신경망을 교차해 지나감
- 모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 ****변환(가중합 및 활성화 함수)****을 적용하여 다음 층(은닉층)의 뉴런으로 전송하는 방식
- 데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 $\rightarrow$ 그 예측 값은 ****최종 층(출력층)****에 도달

◆ 손실 계산

- 손실 함수로 네트워크의 ****예측 값과 실제 값의 차이(손실, 오차)****를 추정
- 손실 함수 비용은 **'0'이 이상적**
- 따라서 손실 함수 비용이 0에 가깝도록 하기 위해 모델이 **훈련을 반복하면서 가중치를 조정**

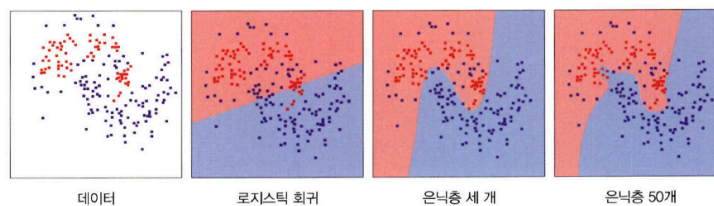
◆ 2단계: 역전파 (Backpropagation)

- 손실(오차)이 계산되면 그 정보는 **역으로 전파(출력층 → 은닉층 → 입력층)** 되기 때문에 역전파라고 함
- 출력층에서 시작된 손실 비용은 **은닉층의 모든 뉴런으로 전파**
- 단, 은닉층의 뉴런은 **각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라(즉, 가중치에 따라) 값이 달라짐**
- **수학적으로**: 예측 값과 실제 값 차이를 각 뉴런의 가중치로 미분한 후 → 기존 가중치 값에서 뺌
- 이 과정을 **출력층 → 은닉층 → 입력층** 순서로 모든 뉴런에 대해 진행
- 계산된 각 뉴런 결과를 **또다시 순전파의 가중치 값으로 사용**

4.2.3 딥러닝의 문제점과 해결방안

- 딥러닝의 핵심=활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현하는 것
- 활성화 함수가 적용된 **은닉층 개수가 많을수록 데이터 분류가 잘됨**

▼ 그림 4-13 은닉층이 분류에 미치는 영향



- 데이터 → 로지스틱 회귀(직선 경계) → 은닉층 세 개(곡선 경계) → 은닉층 50개(데이터 모양에 딱 맞는 경계)
- 은닉층이 많아질수록 빨강/파랑 점을 더 정교하게 나눔

하지만 은닉층이 많을수록 세 가지 문제점 발생

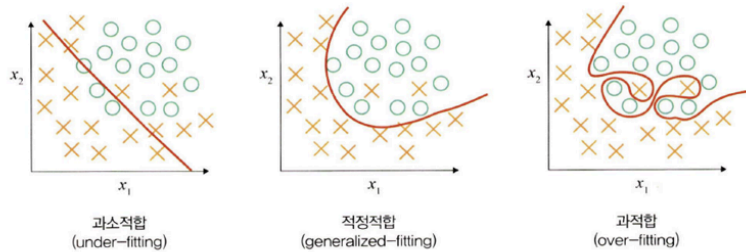
문제	내용	해결 방안
① 과적합	훈련 데이터만 너무 잘 맞춤	드롭아웃
② 기울기 소멸	오차가 입력층까지 못 감	렐루 활성화 함수
③ 성능 저하	경사 하강법이 느리고 불안정	확률적/미니 배치 경사 하강법 + 옵티마이저

① 과적합 문제 발생 → 드롭아웃

과적합 (over-fitting)

- **훈련 데이터를 과하게 학습해서 발생**
- 일반적으로 훈련 데이터는 **실제 데이터의 일부분**
- 훈련 데이터를 과하게 학습했기 때문에 → 예측 값과 실제 값 차이인 **오차는 감소하지만, 검증 데이터에 대해서는 오차가 증가할 수 있음**
- 즉, 훈련 데이터에 대해 과하게 학습하여 **실제 데이터에 대한 오차가 증가하는 현상**

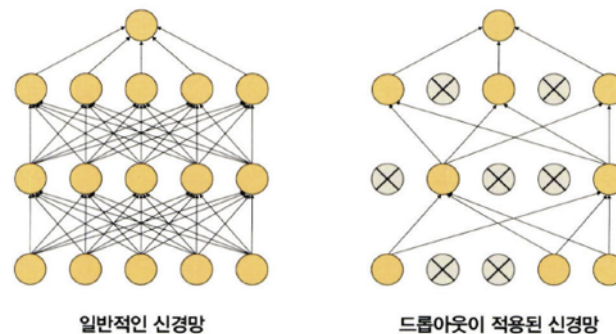
▼ 그림 4-14 과적합



드롭아웃 (dropout)

- 과적합을 해결하는 방법
- 신경망 모델이 과적합되는 것을 피하기 위해, **학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴**

▼ 그림 4-15 일반적인 신경망과 드롭아웃이 적용된 신경망



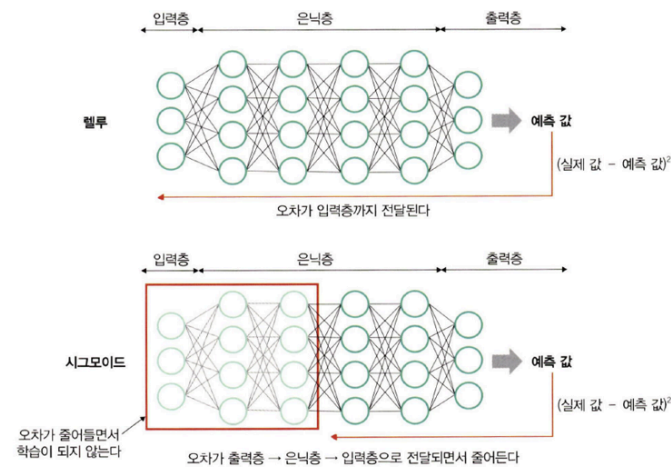
- 일반적인 신경망: 모든 노드가 연결
- 드롭아웃 적용: 층마다 일부 노드에 ⊗ 표시 → 해당 노드는 학습에서 제외, 연결선도 줄어들음

② 기울기 소멸 문제 발생 → 렐루

- **은닉층이 많은 신경망에서 주로 발생**
- **출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상**

- 즉, 기울기가 소멸되기 때문에 학습되는 양이 '0'에 가까워져 → 학습이 더디게 진행된다 오차를 더 줄이지 못하고 **그 상태로 수렴**
- 해결:** 시그모이드나 하이퍼볼릭 탄젠트 대신 **렐루 활성화 함수** 사용

▼ 그림 4-16 기울기 소멸 문제

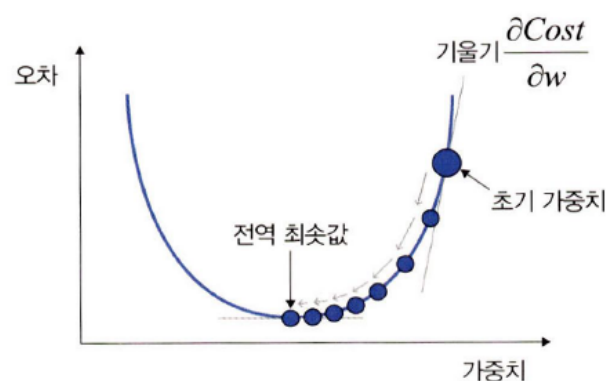


- 렐루:** 오차가 (실제 값 - 예측 값)²에서 출발해 **입력층까지 전달됨**
- 시그모이드:** 오차가 출력층 → 은닉층 → 입력층으로 전달되면서 **점점 줄어듦** → 입력층 쪽 은닉층은 오차가 거의 없어서 **학습이 되지 않음** (그림에서 흐릿하게 표시된 부분)

③ 성능이 나빠지는 문제 발생 → 경사 하강법 개선

- 경사 하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 **기울기가 낮은 쪽으로 계속 이동**시키는 과정을 반복
- 이때 **성능이 나빠지는 문제** 발생

▼ 그림 4-17 경사 하강법



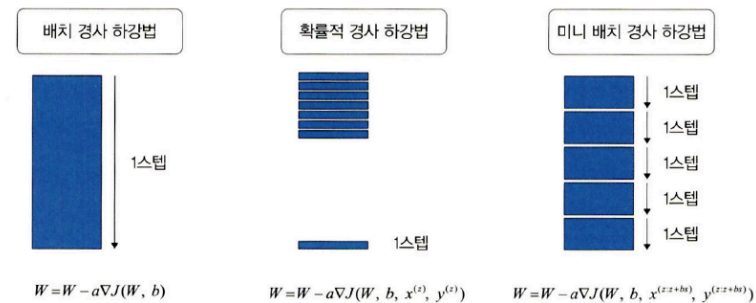
- x축: 가중치 / y축: 오차
- 초기 가중치에서 시작 → 기울기($\partial Cost / \partial w$)를 따라 내려감 → **전역 최솟값**에 도달

- 내려갈수록 이동 폭이 작아짐

- 이러한 문제점을 개선하고자 **확률적 경사 하강법**과 **미니 배치 경사 하강법**을 사용

📌 경사 하강법의 유형

▼ 그림 4-18 경사 하강법의 유형



- 배치 경사 하강법: 전체 데이터 통째로 1스텝
- 확률적 경사 하강법: 데이터 1개씩 1스텝
- 미니 배치 경사 하강법: 데이터 묶음 하나마다 1스텝

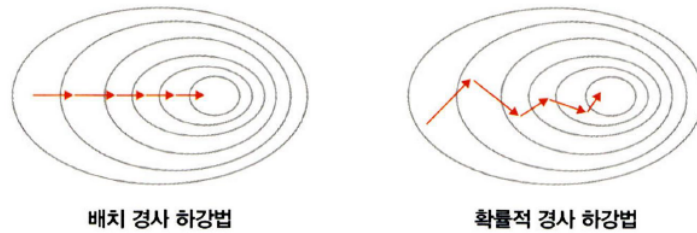
◆ 배치 경사 하강법 (Batch Gradient Descent, BGD)

- 전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트
- 즉, 전체 훈련 데이터셋(total training dataset)에 대해 가중치를 편미분하는 방법
- **단점:** 한 스텝에 모든 훈련 데이터셋을 사용하므로 **학습이 오래 걸림**
- 수식: $W = W - a \nabla J(W, b)$
 - 손실 함수의 값을 최소화하기 위해 기울기(∇) 이용
 - a: 학습률, J: 손실 함수

◆ 확률적 경사 하강법 (Stochastic Gradient Descent, SGD)

- 배치 경사 하강법의 학습이 오래 걸리는 단점을 개선한 방법
- 임의로 선택한 데이터에 대해 기울기를 계산 → 적은 데이터를 사용하므로 **빠른 계산** 가능
- **단점:** 파라미터 변경 폭이 **불안정**하고, 때로는 배치 경사 하강법보다 **정확도가 낮을** 수 있음
- **장점:** 속도가 빠름
- 수식: $W = W - a \nabla J(W, b, x^{(z)}, y^{(z)})$

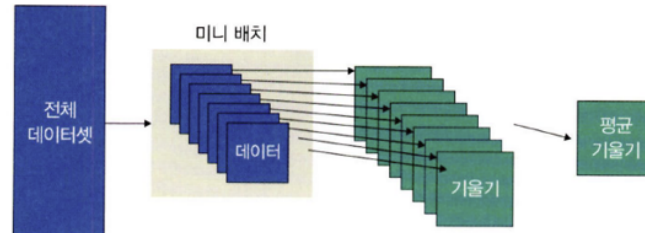
▼ 그림 4-19 배치 경사 하강법과 확률적 경사 하강법



◆ 미니 배치 경사 하강법 (Mini-batch Gradient Descent)

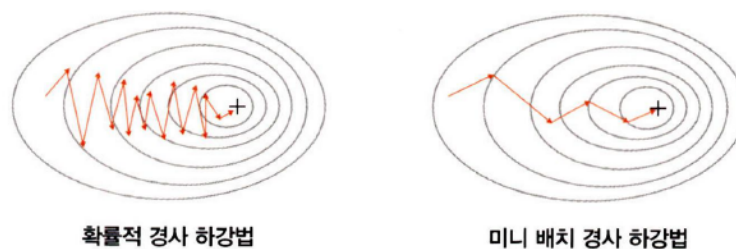
- 전체 데이터셋을 **미니 배치(mini-batch) 여러 개로 나누고**, 미니 배치 한 개마다 기울기를 구한 후 → **그것의 평균 기울기**를 이용하여 모델을 업데이트
- 전체 데이터를 계산하는 것보다 **빠르며**, 확률적 경사 하강법보다 **안정적** → **실제로 가장 많이 사용**
- 파라미터 변경 폭이 확률적 경사 하강법에 비해 안정적이면서 속도도 빠름
- 수식: $W = W - a \nabla J(W, b, x^{(z, z+bs)}, y^{(z, z+bs)})$

▼ 그림 4-20 미니 배치 경사 하강법



- 전체 데이터셋 → 미니 배치 여러 개 (데이터) → 각각 기울기 계산 → **평균 기울기**

▼ 그림 4-21 확률적 경사 하강법과 미니 배치 경사 하강법



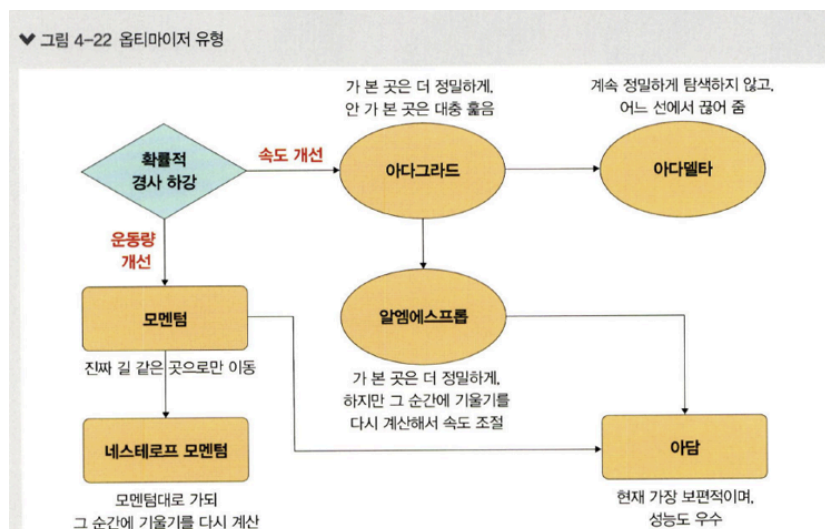
- 확률적: 심하게 지그재그
- 미니 배치: 흔들림이 훨씬 적고 안정적으로 중심에 도달

✓ 경사 하강법 한눈에 정리

구분	1스텝에 쓰는 데이터	속도	안정성	특징
배치 (BGD)	전체	느림	안정적	정확하지만 오래 걸림
확률적 (SGD)	1개 (임의)	빠름	불안정 (지그재그)	정확도 낮을 수 있음
미니 배치	묶음 (배치 크기만큼)	빠름	안정적	실제로 가장 많이 사용

📌 Note: 옵티마이저 (Optimizer)

- 확률적 경사 하강법의 **파라미터 변경 폭이 불안정한 문제**를 해결하기 위해 → **학습 속도와 운동량을 조정**하는 옵티마이저를 적용



- 확률적 경사 하강 에서 출발
- 속도 개선 방향 → 아다그라드 → 아다델타 / 알엠에스프롬
- 운동량 개선 방향 → 모멘텀 → 네스테로프 모멘텀
- 두 방향 합류 → **아담** (현재 가장 보편적이며, 성능도 우수)

옵티마이저	그림 속 한 줄 설명
아다그라드	가 본 곳은 더 정밀하게, 안 가 본 곳은 대충 훑음
아다델타	계속 정밀하게 탐색하지 않고, 어느 선에서 끊어 줌
알엠에스프롬	가 본 곳은 더 정밀하게, 하지만 그 순간에 기울기를 다시 계산해서 속도 조절
모멘텀	진짜 길 같은 곳으로만 이동
네스테로프 모멘텀	모멘텀대로 가되 그 순간에 기울기를 다시 계산
아담	현재 가장 보편적이며, 성능도 우수

◆ 속도를 조정하는 방법

아다그라드 (Adagrad, Adaptive gradient)

- 변수(가중치)의 업데이트 횟수에 따라 학습률을 조정하는 방법
- 많이 변화하지 않는 변수 → 학습률 크게 / 많이 변화하는 변수 → 학습률 작게
- 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 조정, 적게 변화한 변수는 학습률을 크게 하여 빠르게 오차 값을 줄임
- 수식:
 - $w(i+1) = w(i) - \eta / \sqrt{G(i) + \epsilon} \cdot \nabla E(w(i))$
 - $G(i) = G(i-1) + (\nabla E(w(i)))^2$
- 파라미터마다 다른 학습률을 주기 위해 **G 함수** 추가
 - G 값 = 이전 G 값의 누적(기울기 크기의 누적)
 - 기울기가 크면 G 값이 커지기 때문에 → $\eta / \sqrt{G(i)+\epsilon}$ 에서 **학습률(η)은 작아짐**
 - 즉, 파라미터가 많이 학습되었으면 작은 학습률로, 덜 되었으면 개선의 여지가 많기 때문에 높은 학습률로 업데이트
- 파이토치: `optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)` (학습률 기본값 1e-2)
- 단점: 기울기가 0에 수렴하는 문제가 있어 사용하지 않으며, 대신에 **알엠에스프롭** 사용

아다델타 (Adadelata, Adaptive delta)

- 아다그라드에서 **G 값이 커짐에 따라 학습이 멈추는 문제**를 해결하기 위해 등장
- 아다그라드 수식에서 학습률(η)을 ****D 함수(가중치의 변화량(Δ) 크기를 누적한 값)****로 변환 → **학습률에 대한 하이퍼파라미터가 필요하지 않음**
- 수식:
 - $w(i+1) = w(i) - \sqrt{D(i-1) + \epsilon} / \sqrt{G(i) + \epsilon} \cdot \nabla E(w(i))$
 - $G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$
 - $D(i) = \gamma D(i-1) + (1-\gamma)(\Delta w(i))^2$
- 파이토치: `optimizer = torch.optim.Adadelata(model.parameters(), lr=1.0)` (학습률 기본값 1.0)

알엠에스프롭 (RMSProp)

- 아다그라드의 **G(i) 값이 무한히 커지는 것을 방지**하고자 제안된 방법
- 수식:

- $w(i+1) = w(i) - \eta / \sqrt{G(i) + \epsilon} \cdot \nabla E(w(i))$
- $G(i) = \gamma G(i-1) + (1-\gamma)(\nabla E(w(i)))^2$
- 아다그라드에서 학습이 안 되는 문제를 해결하기 위해 G 함수에서 **γ (감마)만 추가**
- G 값이 너무 크면 학습률이 작아져 학습이 안 될 수 있으므로 → 사용자가 **γ 값을 이용하여 학습률 크기를 비율로 조정할 수 있도록 함**
- 파이토치: `optimizer = torch.optim.RMSprop(model.parameters(), lr=0.01)` (학습률 기본값 1e-2)

◆ 운동량을 조정하는 방법

모멘텀 (Momentum)

- 경사 하강법과 마찬가지로 매번 기울기를 구하지만, 가중치를 수정하기 전에 **이전 수정 방향 (+, -)을 참고하여 같은 방향으로 일정한 비율만 수정**
- 수정이 양(+)의 방향과 음(-)의 방향으로 순차적으로 일어나는 **지그재그 현상이 줄어들고**, 이전 이동 값을 고려하여 일정 비율만큼 다음 값을 결정하므로 **관성 효과**를 얻을 수 있음
- SGD(확률적 경사 하강법)와 함께 사용
- **확률적 경사 하강법 수식:** $w(i+1) = w(i) - \eta \nabla E(w(i))$
 - $\eta \nabla E(w(i))$ 수식을 사용하여 가중치 계산 → **기울기 크기와 반대 방향만큼** 가중치 업데이트
 - 즉, 기울기가 크면 아래쪽(-) 방향으로 업데이트
- **SGD 모멘텀 (SGD with Momentum):** 확률적 경사 하강법에서 기울기($\eta \nabla E(w(i))$)를 **속도(v, velocity)**로 대체하여 사용하는 방식 → **이전 속도의 일정 부분을 반영**
 - 즉, 이전에 학습했던 속도와 현재 기울기를 반영해서 가중치를 구함
- 수식:
 - $w(i+1) = w(i) - v(i)$
 - $v(i) = \gamma v(i-1) + \eta \nabla E(w(i))$
- 파이토치: `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)`
 - momentum 값은 **0.9에서 시작**하며 0.95, 0.99처럼 조금씩 증가시키면서 사용

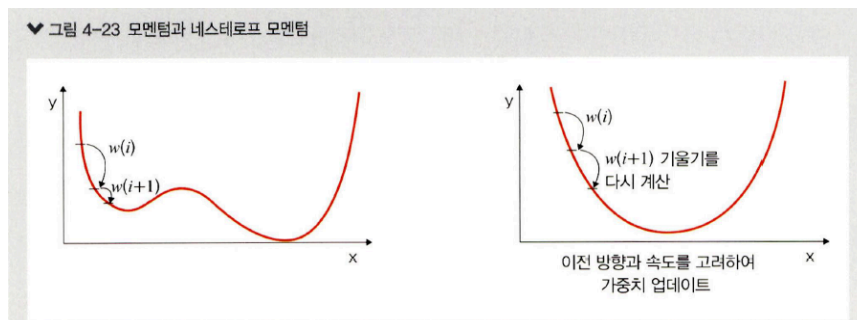
네스테로프 모멘텀 (Nesterov Accelerated Gradient, NAG)

- 기존 모멘텀: 모멘텀 값과 기울기 값이 더해져 실제 값을 만들
- 네스테로프: **모멘텀 값이 적용된 지점에서 기울기 값을 계산**

- 모멘텀 방법의 단점: 멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있음
- 네스테로프: 모멘텀으로 절반 정도 이동한 후 어떤 방식으로 이동해야 하는지 다시 계산하여 결정 → 모멘텀 단점 극복
- 모멘텀의 이점인 빠른 이동 속도는 그대로 가져가면서, 멈추어야 할 적절한 시점에서 제동을 거는 데 훨씬 용이
- 수식:
 - $w(i+1) = w(i) - v(i)$
 - $v(i) = \gamma v(i-1) + \eta \nabla E(w(i) - \gamma v(i-1))$
- 모멘텀과 비슷하지만 속도(v)를 구하는 과정에서 차이: 이전에 학습했던 속도와 현재 기울기에서 이전 속도를 뺀 변화량을 반영해서(더해서) 가중치를 구함
- 파이토치: `optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)` (nesterov 기본값 False)

🔍 모멘텀과 네스테로프 모멘텀

- 모멘텀: $w(i) \rightarrow w(i+1)$ 로 관성 따라 이동, 곡선의 작은 골짜기를 지나쳐 갈 수 있음
- 네스테로프: $w(i)$ 에서 일단 이동한 뒤 $w(i+1)$ 지점에서 기울기를 다시 계산 → 이전 방향과 속도를 고려하여 가중치 업데이트



◆ 속도와 운동량에 대한 혼용 방법

아담 (Adam, Adaptive Moment Estimation)

- 모멘텀과 알엠에스프롬의 장점을 결합한 경사 하강법
- 알엠에스프롬 특징인 기울기의 제곱을 지수 평균한 값(G 함수) + 모멘텀 특징인 $v(i)$ 를 수식에 활용
- 수식:
 - $w(i+1) = w(i) - \eta / \sqrt{G(i) + \epsilon} \cdot v(i)$

- $G(i) = \gamma_2 G(i-1) + (1-\gamma_2)(\nabla E(w(i)))^2$
- $v(i) = \gamma_1 v(i-1) + \eta \nabla E(w(i))$
- 파이토치: `optimizer = torch.optim.Adam(model.parameters(), lr=0.01)` (학습률 기본값 1e-3)

✔ 옵티마이저 정리표

옵티마이저	조정 대상	핵심 아이디어	문제점 / 비교	파이토치 (lr 기본값)
아다그라드	속도	많이 변한 변수는 학습률 ↓, 적게 변한 변수는 ↑ (G = 기울기 ² 누적)	G가 계속 커져 학습률 → 0, 학습 멈춤	<code>Adagrad</code> (1e-2)
아다델타	속도	학습률 η 대신 D 함수(변화량 누적) 사용	학습률 하이퍼파라미터 불필요	<code>Adadelta</code> (1.0)
알엠에스프롭	속도	G 함수에 γ 추가 → G 무한 증가 방지	아다그라드 대신 사용	<code>RMSprop</code> (1e-2)
모멘텀	운동량	이전 속도 v를 일정 비율(γ) 반영 → 관성 효과, 지그재그 감소	멈춰야 할 때도 관성으로 지나침	<code>SGD (momentum=0.9)</code>
네스테로프	운동량	모멘텀으로 먼저 이동한 지점에서 기울기 재계산	제동 걸기 용이	<code>SGD (momentum=0.9, nesterov=True)</code>
아담	속도 + 운동량	알엠에스프롭의 G + 모멘텀의 v 결합	현재 가장 보편적, 성능 우수	<code>Adam</code> (1e-3)



4.2.4 딥러닝을 사용할 때 이점

◆ 특성 추출 (Feature Extraction)

- 컴퓨터가 입력받은 데이터를 분석하여 일정한 패턴이나 규칙을 찾아내려면 → **사람이 인지하는 데이터를 컴퓨터가 인지할 수 있는 데이터로 변환**해 주어야 함
- 이때 데이터별로 **어떤 특징을 가지고 있는지 찾아내고**, 그것을 토대로 데이터를 벡터로 변환하는 작업 = **특성 추출(feature extraction)**

딥러닝 이전 vs 딥러닝

구분	딥러닝 이전 (머신 러닝)	딥러닝
대표 알고리즘	SVM, 나이브 베이즈(Naïve Bayes), 로지스틱 회귀	심층 신경망
특성 추출	사람이 직접 → 매우 복잡	알고리즘에 통합 (모델이 알아서)
필요 조건	수집된 데이터에 대한 전문 지식 필요 (예: 제조, 의료 등 도메인 지식)	도메인 지식 없이도 가능
가능한 이유	-	데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조 덕분

◆ 빅데이터의 효율적 활용

- 딥러닝에서 특성 추출을 알고리즘에 통합시킬 수 있었던 이유 = **빅데이터** 때문
- 딥러닝 학습을 이용한 특성 추출은 **데이터 사례가 많을수록 성능이 향상**됨
- 반대로 말하면, **확보된 데이터가 적다면** 딥러닝의 성능 향상을 기대하기 힘들기 때문에 → **머신러닝을 고려**해 보아야 함

✓ 정리

데이터 양	적합한 방법	이유
많음 (빅데이터)	딥러닝	데이터가 많을수록 특성 추출 성능 ↑
적음	머신 러닝	딥러닝은 데이터가 적으면 성능 향상 기대 어려움

4.3 딥러닝 알고리즘

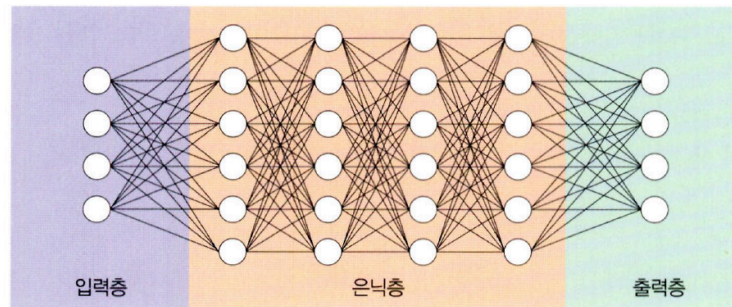
- 딥러닝 알고리즘은 **심층 신경망을 사용**한다는 공통점이 있음
- 머신 러닝 알고리즘처럼 목적에 따른 분류: 합성곱 신경망(CNN), 순환 신경망(RNN), 심층 신뢰 신경망(DBN)

4.3.1 심층 신경망 (Deep Neural Network, DNN)

- **입력층과 출력층 사이에 다수의 은닉층을 포함**하는 인공 신경망

- 머신 러닝에서는 비선형 분류를 하기 위해 여러 트릭 (trick)을 사용 → 심층 신경망은 **다수의 은닉층을 추가했기 때문에 별도의 트릭 없이 비선형 분류 가능**
- 장점: 다수의 은닉층을 두었기 때문에 **다양한 비선형적 관계를 학습**할 수 있음
- 단점: 학습을 위한 **연산량이 많고, 기울기 소멸 문제** 등이 발생할 수 있음
- 해결: 앞서 설명한 **드롭아웃, 렐루 함수, 배치 정규화** 등을 적용

♥ 그림 4-24 심층 신경망

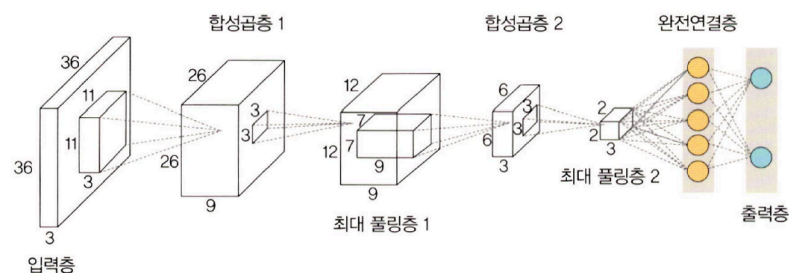


- 입력층(4개 노드) → 은닉층(4개 층, 각 6개 노드) → 출력층(4개 노드)
- 모든 노드가 다음 층의 모든 노드와 연결됨

4.3.2 합성곱 신경망 (Convolutional Neural Network. CNN)

- 합성곱층(convolutional layer)과 풀링층(pooling layer)을 포함하는 **이미지 처리 성능이 좋은** 인공 신경망 알고리즘
- 영상 및 사진이 포함된 이미지 데이터에서 **객체를 탐색하거나 객체 위치를 찾아내는 데 유용**
- 이미지에서 **객체, 얼굴, 장면을 인식**하기 위해 패턴을 찾는 데 특히 유용
- **대표적인 합성곱 신경망**: LeNet-5, AlexNet
- **층을 더 깊게 쌓은 신경망**: VGG, GoogLeNet, ResNet 등

♥ 그림 4-25 합성곱 신경망



- 입력층($36 \times 36 \times 3$) → 합성곱층 1($26 \times 26 \times 3$) → 최대 풀링층 1($12 \times 12 \times 3$) → 합성곱층 2($6 \times 6 \times 3$) → 최대 풀링층 2($2 \times 2 \times 3$) → 완전연결층 → 출력층

- 합성곱 → 풀링을 반복하며 이미지 크기는 줄고 특징만 남김 → 마지막에 완전연결층으로 분류

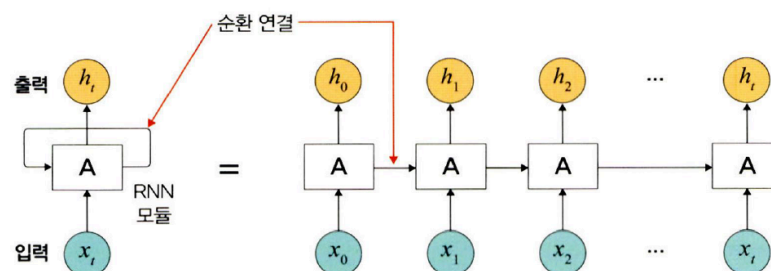
기존 신경망과 비교한 차별성

- 각 층의 **입출력 형상을 유지**
- 이미지의 **공간 정보를 유지**하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- **복수 필터**로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 **풀링층**이 있음
- 필터를 **공유 파라미터**로 사용하기 때문에 일반 인공 신경망과 비교하여 **학습 파라미터가 매우 적음**

4.3.3 순환 신경망 (Recurrent Neural Network, RNN)

- **시계열 데이터(음악, 영상 등)** 같은 **시간 흐름에 따라 변화하는 데이터**를 학습하기 위한 인공 신경망
- **순환(recurrent)** = 자기 자신을 참조한다는 것 → **현재 결과가 이전 결과와 연관**이 있다는 의미

▼ 그림 4-26 순환 신경망



순환 신경망의 특징

- **시간성 (temporal property)**을 가진 데이터가 많음
- 시간성 정보를 이용하여 데이터의 특징을 잘 다룸
- 시간에 따라 내용이 변하므로 데이터는 **동적**이고, **길이**가 **가변적**
- 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음

문제점과 해결

- **기울기 소멸 문제(vanishing gradient problem)**로 학습이 제대로 되지 않는 문제가 있음
- 이를 해결하고자 메모리 개념을 도입한 **LSTM(Long-Short Term Memory)**이 순환 신경망에서 많이 사용됨

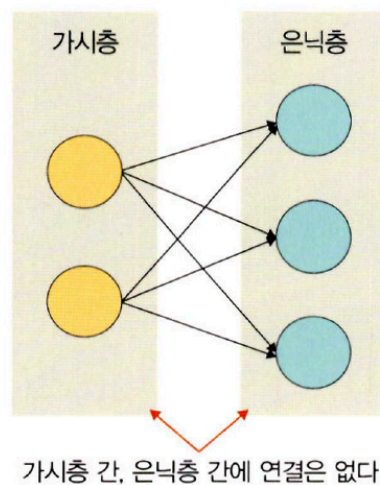
활용 분야

- 자연어 처리 분야와 궁합이 맞음
- 대표적인 예: 언어 모델링, 텍스트 생성, 자동 번역(기계 번역), 음성 인식, 이미지 캡션 생성 등

4.3.4 제한된 볼츠만 머신 (Restricted Boltzmann Machine, RBM)

- 볼츠만 머신(Boltzmann machine): 가시층(visible layer)과 은닉층(hidden layer)으로 구성된 모델
- 이 모델에서 가시층은 은닉층과만 연결됨 (가시층과 가시층, 은닉층과 은닉층 사이에 연결은 없음) → 이것이 제한된 볼츠만 머신 (RBM)

▼ 그림 4-27 제한된 볼츠만 머신



- 가시층(2개 노드) ↔ 은닉층(3개 노드) 사이만 연결
- 가시층 간, 은닉층 간에 연결은 없다

제한된 볼츠만 머신의 특징

- *차원 감소, 분류, 선형 회귀 분석, 협업 필터링(collaborative filtering), 특성 값 학습(feature learning), 주제 모델링(topic modelling)**에 사용
- 기울기 소멸 문제를 해결하기 위해 **사전 학습 용도**로 활용 가능
- **심층 신뢰 신경망(DBN)의 요소**로 활용됨

→ 딥러닝에서 많이 사용되는 알고리즘은 **CNN**과 **RNN**. 제한된 볼츠만 머신과 심층 신뢰 신경망은 상대적으로 많이 사용하지 않음

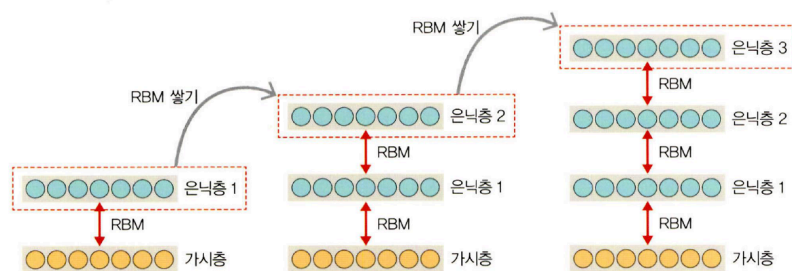
4.3.5 심층 신뢰 신경망 (Deep Belief Network, DBN)

- 입력층과 은닉층으로 구성된 **제한된 볼츠만 머신**을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망
- 즉, **사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조**
- 레이블이 없는 데이터에 대한 **비지도 학습**이 가능
- 부분적인 이미지에서 전체를 연상하는 **일반화와 추상화 과정**을 구현할 때 사용하면 유용

심층 신뢰 신경망의 학습 절차

1. 가시층과 은닉층 1에 제한된 볼츠만 머신을 **사전 훈련**
2. 첫 번째 층 입력 데이터와 파라미터를 고정하여 두 번째 층 제한된 볼츠만 머신을 사전 훈련
3. 원하는 층 개수만큼 제한된 볼츠만 머신을 **쌓아 올려** 전체 DBN을 완성

▼ 그림 4-28 심층 신뢰 신경망



- 1단계: 가시층 ↔ 은닉층 1 (RBM)
- 2단계: RBM 쌓기 → 가시층 ↔ 은닉층 1 ↔ 은닉층 2
- 3단계: RBM 쌓기 → 가시층 ↔ 은닉층 1 ↔ 은닉층 2 ↔ 은닉층 3
- 각 층 사이가 하나의 RBM

심층 신뢰 신경망의 특징

- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 **계층적 구조**를 생성
- **비지도 학습**으로 학습
- 위로 올라갈수록 **추상적 특성**을 추출
- 학습된 가중치를 **다층 퍼셉트론의 가중치 초기값**으로 사용

✓ 딥러닝 알고리즘

알고리즘	구조 특징	잘하는 것	문제점 / 해결	사용 빈도
DNN	은닉층 다수	비선형 분류 (트릭 없이)	연산량 많음, 기울기 소멸 → 드롭아웃·렐루·배치 정규화	기본

알고리즘	구조 특징	잘하는 것	문제점 / 해결	사용 빈도
CNN	합성곱층 + 풀링층	이미지 (객체·얼굴·장면 인식)	-	많이 사용
RNN	자기 자신을 참조 (순환)	시계열·자연어 (번역, 음성 인식 등)	기울기 소멸 → LSTM	많이 사용
RBM	가시층 ↔ 은닉층만 연결	차원 감소, 협업 필터링, 사전 학습	-	적음
DBN	RBM을 층층이 쌓음	비지도 학습, 추상적 특성 추출	-	적음

4.4 우리는 무엇을 배워야 할까?

- 머신 러닝을 학습할지, 딥러닝을 학습할지는 **주어진 데이터를 활용하여 어떤 결과를 얻고 싶은지**에 따라 다름

상황	선택
간단한 선형 회귀 분류 를 이용하여 원하는 값을 도출할 수 있다면	머신 러닝 만으로도 충분
복잡한 비선형 데이터 에 대한 분류 및 예측을 도출하고 싶다면	딥러닝 으로 학습

- 즉, 머신 러닝과 딥러닝은 우리가 얻고 싶은 결과를 도출하기 위한 ****도구(연장)****일 뿐
- 어떤 도구를 선택했을 때 더 좋은 성능과 효과를 얻을 수 있을지는 **데이터를 수집하고 분석하는 사람만 알 수 있음**
- 데이터를 활용하여 얻고자 하는 것에 따라 머신 러닝이나 딥러닝을 **선택해서 학습하고 데이터를 훈련**시키면 됨